~/majibill/docs/11-auth-onboarding-rbac.md

$ cat 11-auth-onboarding-rbac.md

MAJIBILL PLATFORM · IDENTITY

Authentication, Onboarding, Tenant Flow & RBAC Dashboard Parity

How people sign in, how tenants and users are onboarded, how a tenant moves through its lifecycle, and how one permission model drives every dashboard identically.

```
can(user, 'bill:approve', {tenant, zone})
// one permission model: API, web, mobile, dashboards
// the UI hides, the API enforces
```

DOC	VERSION	DATE
MB-11	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Backend, frontend, mobile, security, product	Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app
Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

1. Scope and principles . . . 3
2. Realms and actors . . . 3
3. Authentication flows . . . 3
4. Onboarding . . . 5
5. Tenant lifecycle . . . 6
6. RBAC model . . . 6
7. Dashboard parity . . . 7
8. Implementation checklist . . . 8

// 1. Scope and principles

Identity is where tenancy, security and usability meet. This document defines three realms, the sign-in flows for each, the end-to-end onboarding of a partner, tenant and its staff, the tenant lifecycle, and the rule that makes dashboards consistent: **one permission manifest drives the API, the web apps and the mobile app**.

1. **Three realms, never mixed:** Platform (404HubSpot), Partner (ZUCAR), Tenant (utility staff and customers). A token is valid in exactly one realm.
2. **Enforce on the server, reflect in the UI.** Hiding a button is a convenience; the API guard is the control.
3. **Parity by construction.** Dashboards are assembled from a role's permissions and data scope, not hand-built per role.
4. **Self-service where safe, approved where risky.** Inviting a clerk is self-service; granting admin needs a second admin.
5. **Every identity event is audited** (MB-06).

// 2. Realms and actors

Realm	Actors	Sign-in	Data scope
Platform	404HubSpot admins, support, finance, security	SSO-capable staff login + mandatory MFA; device trust; IP allow-list for console	Control-plane tables; tenant data only via break-glass sessions
Partner	ZUCAR admins and sales/support staff	Email + password + MFA	Their tenants' health, usage, invoices; no customer PII by default
Tenant staff	Admin, billing manager, clerk, cashier, reader, auditor	Email/phone + password + MFA (staff); reader app uses device-bound login + PIN	One tenant, optionally narrowed to zones or routes
Customer	Water customers and their delegates	Phone number + SMS OTP (no password); optional PIN	Own accounts only
Machine	API keys, integrations, ingress service	Scoped API key / signed requests	Explicit scopes per key

// 3. Authentication flows

3.1 Staff sign-in (web)

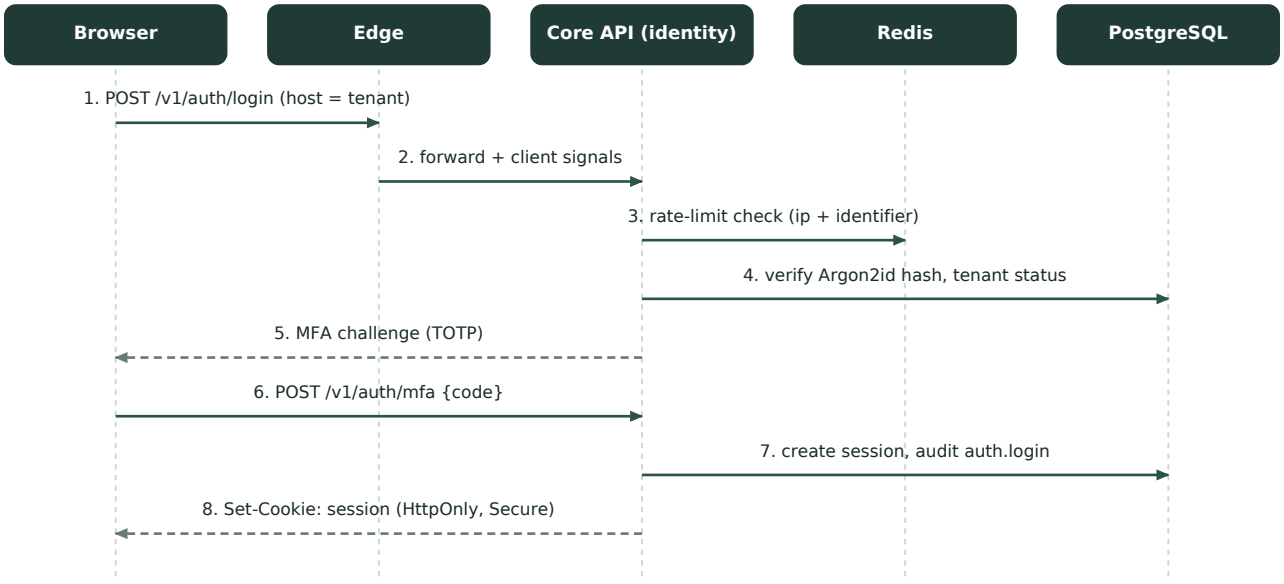


Fig 3.1 Tenant is resolved from the host before credentials are checked, so a user can only ever sign in to their own tenant.

3.2 Customer sign-in (OTP) and reader app

Step	Customer portal	Reader app
1	Enter phone number; rate-limited; generic response to avoid account enumeration	Admin invites reader; device registers with a one-time enrolment code
2	SMS OTP (6 digits, 5 min, 5 attempts, single use); resend throttled	Reader sets a PIN; device key stored in secure storage
3	Session cookie; shows accounts linked to this phone; step-up OTP before payment or profile change	Short-lived access token + rotating refresh; works offline until refresh expiry (configurable, default 7 days)
4	Account linking: first link needs account number + a recent-bill detail, preventing guessing	Remote revoke from console wipes cached data on next contact

3.3 Tokens and sessions

● ● ● token model

```
access token (JWT, 10 min):  sub, realm, tenant_id, roles[], scope{zones[],routes[]},
                             perm_v (permission set version), sid, amr (pwd|mfa|otp)
refresh / session:           opaque id, stored server-side, rotated on use, bound to device
revocation:                  session table + perm_v bump; guards compare perm_v to cached policy
step-up:                     sensitive actions require amr contains 'mfa' within last 10 min
```

NOTE **Why** `perm_v`

When a role changes, the tenant’s permission version increments. Tokens carrying an older version are re-evaluated on the next request, so revocations take effect in seconds without waiting for expiry.

// 4. Onboarding

Onboarding is a **workflow with checkpoints**, executed by the platform console (and later self-serve). It is idempotent: each step can be re-run safely.

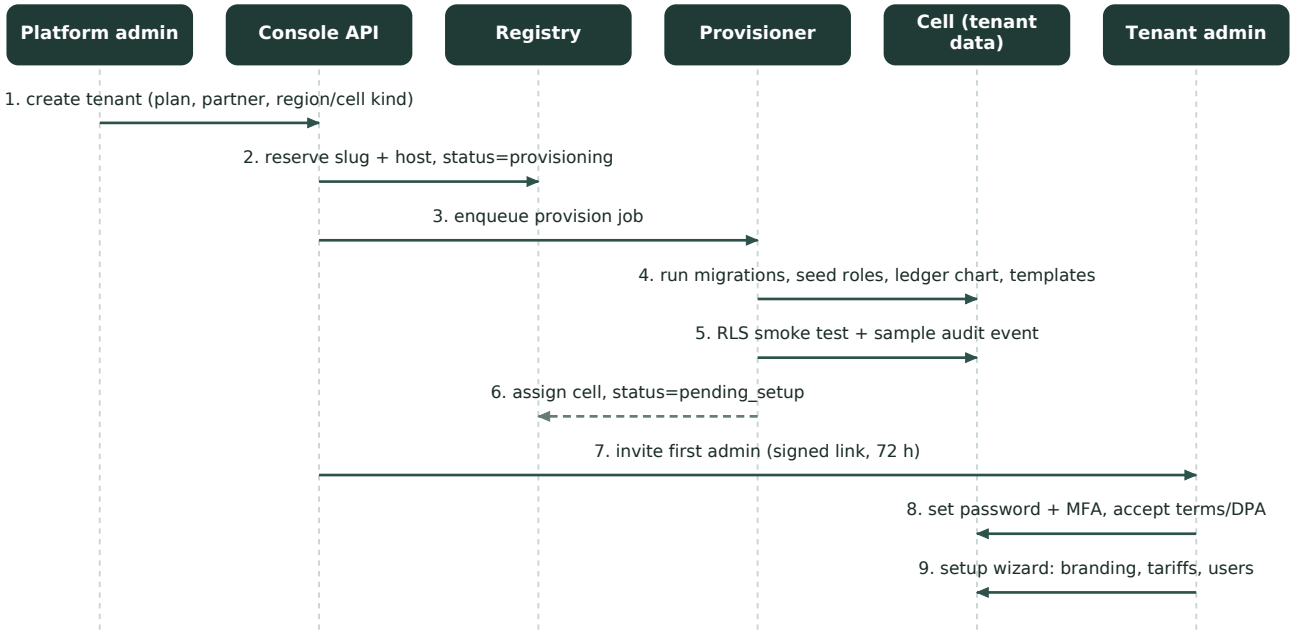


Fig 4.1 Tenant provisioning. The first admin is invited, not created with a shared password.

4.1 Setup wizard (tenant admin)

Step	Collects	Done when
1 Organisation	Legal name, logo, colours, contact details, SMS sender name	Saved and previewed on a sample bill
2 Payments	Paybill/Till, Daraja credentials (stored in the secrets manager), test payment	Sandbox or live test payment posts
3 Structure	Zones, routes, customer classes	At least one route
4 Tariffs	Tariff plans and charge types with effective dates	Sample calculation approved
5 Users and roles	Invite staff; assign roles and scopes	Admin + one operational role invited
6 Data import	Customers, accounts, meters, opening balances (templates + validation report)	Import dry-run passes; reconciliation total matches
7 Go-live checklist	Notification templates, approvals thresholds, retention, DPA accepted	All checks green; platform admin signs off

4.2 User invitation and role changes

- Invitations are single-use signed links with expiry; the invitee sets credentials and MFA; the inviter never sees the password.
- Granting `tenant_admin`, changing approval thresholds, or removing the last admin requires a second admin (maker–checker) and notifies all admins.
- Offboarding: disabling a user revokes sessions and devices immediately, reassigns open tasks, and keeps their audit history.
- Bulk user import supports CSV with validation; roles must exist before assignment.

// 5. Tenant lifecycle

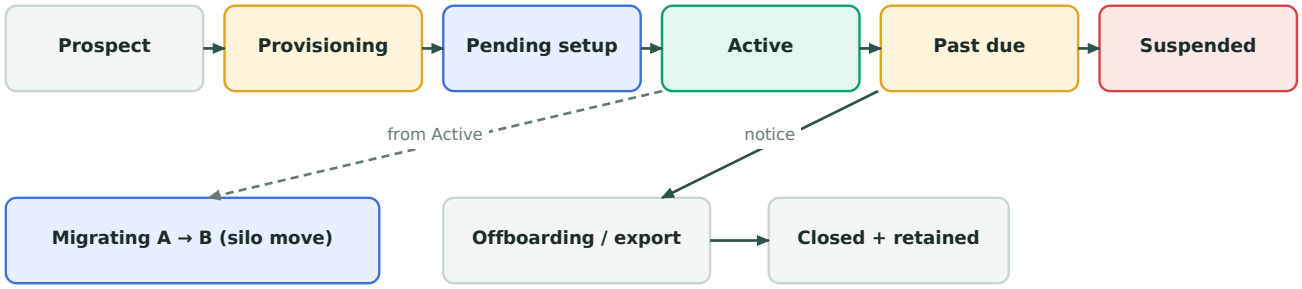


Fig 5.1 Tenant lifecycle. Suspension blocks sign-in and new bills but never deletes data; billing runs and payments already in flight complete safely.

State	Meaning	Effects
Provisioning / Pending setup	Cell resources and admin setup	No customer-facing activity
Active	Live and in good standing	Full entitlements
Past due	Subscription payment overdue (MB-12)	Warnings; features unchanged during grace period
Suspended	Non-payment, security hold, or contract breach	Staff sign-in blocked (read-only for admin export), **customer payments still post**, SMS paused
Migrating	Moving between cells	Short read-only window; verified by ledger invariants
Offboarding	Exit requested	Full export, retention timer starts, DPA obligations apply
Closed	Contract ended	Data deleted or anonymised after the retention period; audit anchors kept

> DECISION **Suspension must never lose a customer’s money**
> If a utility’s subscription lapses, customers’ M-Pesa payments to the utility’s Paybill still arrive. The platform keeps ingesting and posting them; only staff features are restricted. This protects customers and the platform’s reputation.

// 6. RBAC model

Concept	Definition
Permission	`resource:action`, e.g. `bill:approve`, `payment:reverse`, `customer:pii_view`
Role	Named set of permissions; system roles are fixed, tenants may clone and adjust within platform limits
Scope	Narrows where a role applies: whole tenant, zone(s), route(s), or own records
Constraint	Rules such as maker–checker, amount thresholds, time windows, requires-MFA
Entitlement	Plan-level feature flags (e.g. reader app, STK push, API access) that gate modules per tenant (MB-12)

policy

```
// one decision function used by API guards, UI manifests and mobile
can(principal, 'adjustment:approve', { tenantId, zoneId, amountMinor }) =>
  principal.realm === 'tenant'
  && entitlements.has('billing')
  && principal.permissions.has('adjustment:approve')
  && inScope(principal.scope, { zoneId })
  && amountMinor <= principal.limits.adjustment_approve
  && principal.id !== resource.requestedBy        // maker-checker
  && recentMfa(principal, 10)                      // step-up
```

// 7. Dashboard parity

Parity means: the same role sees the same numbers and the same actions everywhere (web, mobile, partner view, API), differing only in layout. This prevents the classic failure where the mobile app, the web app and the reports each disagree because each re-implemented the rules.

7.1 How parity is achieved

1. **Permission manifest:** the API exposes `GET /v1/me/manifest` returning permissions, scope, entitlements and feature flags for the signed-in principal.
2. **Widget registry:** each dashboard widget declares `requires: permission[]`, `scope-aware: bool`, and its data endpoint. The UI renders only widgets the manifest satisfies.
3. **Server-side scoping:** widget endpoints apply the same scope filters; a zone supervisor's total is the sum of their zones, computed once by the API.
4. **Shared components and queries:** one set of query definitions and one UI package serve staff web, partner view and platform console; the reader app consumes the same endpoints and manifest.
5. **Parity tests:** CI renders each role's dashboard from fixtures and asserts widget lists and totals against a golden file; a change to a role's permissions must update the golden.

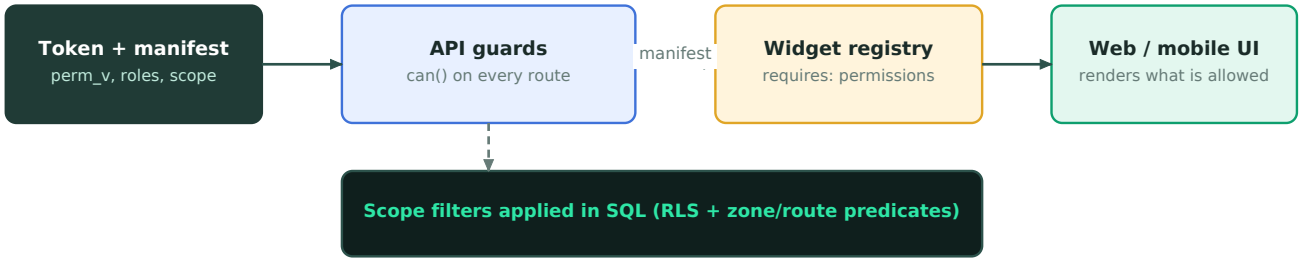


Fig 7.1 One decision path. The UI cannot show what the API would refuse, and the API refuses what the UI hides.

7.2 Dashboard by role

Widget / view	Platform	Partner	Tenant admin	Billing mgr	Cashier	Reader	Auditor	Customer
Fleet health, cells, uptime	Y							
Tenants, plans, usage, MRR	Y	own						
Collections and billed vs collected		agg	Y	Y	own		view	
Arrears ageing		agg	Y	Y			view	
Flagged readings queue			Y	Y		route	view	
Route progress			Y	Y		own		
Suspense payments			Y	Y	Y		view	
Pending approvals			Y	Y				
Audit and access log	Y		Y	Y			Y	
Balance, bills, receipts, usage								own

agg = aggregated counts and totals only, no customer PII. own = limited to the user's own records or routes. view = read-only. Matrix is the default; tenants adjust within platform limits.

7.3 Edge cases to test

- User holds two roles with different scopes: permissions union, scopes union per permission, never per role.
- Zone reassignment mid-session: `perm_v` bump forces re-evaluation.
- Entitlement removed (plan downgrade): widget disappears and API returns `403 entitlement_required`; data stays.

- Partner viewing a suspended tenant: aggregates only, flagged status.
- Support session (break-glass): mirrors the tenant admin view read-only, banner shown, all views audited.
- Mobile offline: manifest cached with expiry; sensitive actions require online re-check.

// 8. Implementation checklist

Area	Deliverable
Identity module	Realms, Argon2id, TOTP, sessions, OTP, API keys, device enrolment, rate limits, audit events
Policy library	`can()` shared package (TS + generated Dart and Go checks from a policy schema)
Manifest API	`/v1/me/manifest` with caching and `perm_v`
Widget registry	Typed registry in `packages/ui`; widget tests; golden dashboards per role
Onboarding	Provisioning workflow, wizard, invitation service, import validators
Admin UX	Role editor with permission diff and impact preview; approval requests; session and device management
Tests	Cross-realm token rejection, scope leaks, privilege escalation attempts, maker–checker bypass, OTP abuse, enumeration